LING285-L9

Course: LING285: Human Language and Technology
Title of Lecture: Neural Networks 1
Date: 9/22/2026
Professor: Khalil Iskarous

Section 1: Differential Equations and Dynamical Systems

- A **dynamical system** is anything in the world whose state changes over time according to a rule. Examples include a spring, a cooling room, or a learning computer program. A **differential equation** is the mathematical way of writing down that rule. It does not directly tell us where something is. It tells us how fast it is changing and in what direction, given where it is right now.
    - Once we know the rule for change, we can work out the whole history of the system, step by step, starting from a known starting point.
    - This is why differential equations are called the **mathematical expression of dynamical systems**. The equation is the rule, and the system's behavior over time is what the rule produces.
- In **1676, Isaac Newton wrote a letter to Gottfried Leibniz** and hid one of his central ideas inside an **anagram**. An anagram in this sense is a scrambled set of letters that works like a code. Newton could send it, and later, once he had fully proven the idea, he could reveal what it said.
    - This was how scientists of that time **established priority**, meaning proof of who had an idea first. If a dispute came up later, Newton could point to the dated letter and show that the scrambled letters spelled out his idea.
    - Newton and Leibniz both developed calculus, the mathematics of change, so establishing who came first mattered a great deal to them.
- Newton's secret, written as the anagram, was **6accdae13eff7i3l9n4o4qrr4s8t12ux**. The numbers count how many times each letter appears. For example, "6a" means six a's, "13e" means thirteen e's, and so on. Together, those letters spell out a Latin sentence.
    - The Latin sentence is: **"Data aequatione quotcunque fluentes quantitates involvente fluxiones invenire, et vice versa."**
    - In English, it means roughly: given an equation involving any number of changing quantities, find their rates of change, and the reverse. Newton called changing quantities **"fluents"** and their rates of change **"fluxions."** A fluxion is what we now call a derivative.
    - The phrase "and vice versa" matters. It means going both ways: from a quantity to its rate of change, and from a rate of change back to the quantity itself. That second direction, rebuilding the quantity from its rule of change, is what solving a differential equation means.
- The mathematician **Vladimir Arnold** paraphrased Newton's secret in a much simpler way: **the laws of nature are expressed by differential equations.** Physical laws describe how things change from moment to moment. If we know the change rule and the starting point, we can predict what happens next.
- **Inference and learning in large language models (LLMs) are also differential equations.** When an LLM processes an input to reach a decision, it passes numbers through a sequence of steps. When it learns, its internal numbers change a little at a time according to a rule. Both processes are rules for change applied step by step, which is the same kind of object Newton described.

Section 2: The Viscous Spring and Negative Feedback

- A **viscous spring** is a spring that moves through something thick, like oil or honey. When it is stretched and released, it does not bounce back and forth. It slides smoothly back toward its resting position and slows down as it gets closer. Its behavior is described by this differential equation:

$$\dot{x} = -\frac{1}{3}x$$

- The symbol x is the current value of the quantity, for example how far the spring is stretched from rest.
- The dot on top, $\dot{x}$, is just **the change in** x. It tells us how much x is changing at this moment.
- The equation says that **the change in** x **is** $-\frac{1}{3}$ **of its current value.**
- This equation is an example of **negative feedback**, also called **balancing feedback**. The minus sign means the change always points opposite to where x currently is. If x is positive, the change is negative, so x shrinks back toward zero. If x were negative, the change would be positive, and x would again move back toward zero.
    - The size of the change also depends on the size of x. When x is large, the push back toward zero is large. When x is small, the push is small. This makes the system move quickly at first and then slow down as it approaches balance.
    - Balancing feedback is what keeps a system near a stable resting point instead of running away.
- The purpose of a differential equation is to take the rule for change, together with an **initial condition**, and figure out x for all time. The initial condition is the value of x at the start. The full description of x at every moment is called **the solution to the differential equation.**
    - The solution is not just a number on paper. It is a **phenomenon in the world**. Examples include **SGD** (the method neural networks use to learn), **flesh** springing back after being pressed, and the **temperature in a room** settling toward a steady value.
    - Each of these follows a rule for change, and the solution describes how it actually behaves over time.

Section 3: Solving the Equation Step by Step: Euler's Method

- We set the initial condition at $x = 100$, meaning that at time $t = 0$, the value of x is 100. The equation tells us that the change in x is $-\frac{1}{3}$ of whatever x currently is.
- At $t = 0$, x is 100. One third of 100 is about 33, so the change is about -33.

$$\dot{x} = -\frac{1}{3}(100) \approx -33.33$$

- Starting at 100 and losing about 33 leaves about 66.

$$x_1 = 100 - 33.33 = 66.67$$

- What we just did can be written as a general rule. The new value of x equals the old value of x minus one third of the old value:

$$x_t = x_{t-1} - \frac{1}{3}x_{t-1}$$

- Here x_t is the value at the current time step, and x_{t-1} is the value at the step before it.
- This rule turns the differential equation into plain arithmetic. We take the current value, compute the change, add the change to the current value, and repeat.

- Now x is about 66. We plug it into the equation again. One third of 66.67 is about 22.22, and we take that away from 66.67:

$$x_2 = 66.67 - 22.22 = 44.44$$

- We repeat the same arithmetic for the next steps. One third of 44.44 is about 14.81:

$$x_3 = 44.44 - 14.81 = 29.63$$

- One third of 29.63 is about 9.88:

$$x_4 = 29.63 - 9.88 = 19.75$$

- Continuing the same way through $t = 8$ gives about 13.17 at $t = 5$, 8.78 at $t = 6$, 5.85 at $t = 7$, and 3.90 at $t = 8$.
  - When these values are plotted with x on the vertical axis and t on the horizontal axis, from $t = 0$ to $t = 8$, the points start high at 100 and drop steeply at first.
  - The drops get smaller and smaller with each step, so the points flatten out and creep closer to the horizontal axis without crossing it. This is exactly the balancing-feedback shape: fast change far from zero, slow change near zero.
- This procedure is called **Euler's Method**. It solves a differential equation by breaking time into small steps and using simple arithmetic at each one.
  - At each step, the equation gives the change based on the current value.
  - Adding that change to the current value produces the next value.
  - Repeating this over and over builds the entire solution, one step at a time.
  - This is how **differential equations are solved to predict what the world is like using arithmetic.** No advanced math is needed at each step, only multiplication and subtraction, repeated many times.

Section 4: Fast and Slow Balancing: Fat and Muscle

- The number multiplying x in a balancing-feedback equation controls **how fast** x returns to zero. Two equations with different numbers produce two curves with the same basic shape but very different speeds.
- The first equation is the one we already solved:

$$\dot{x} = -\frac{1}{3}x$$

- Its solution starts high and falls gradually. It is still clearly above zero far along the time axis and only slowly flattens toward the horizontal axis.
- This slow curve is labeled **fat**. Fat is soft flesh that, once pressed, springs back slowly.
- The second equation has a much larger number:

$$\dot{x} = -3x$$

- Here the change is three times the current value instead of one third of it, so the push back toward zero is nine times stronger at the same value of x.
- Its solution drops almost straight down right after the start, then quickly flattens out and hugs the horizontal axis for the rest of the time.
- This fast curve is labeled **muscle**. Muscle is firmer flesh that snaps back to its resting shape quickly.
- Both fat and muscle behave like **viscous springs** with **negative feedback**. Both return to rest without bouncing. The only difference between them is the size of the number in the

equation. A small number gives a slow, drawn-out return. A large number gives a fast, sharp return.

Section 5: Euler's Method in Python

- Euler's Method is repeated arithmetic, so a computer can do it easily. In Python, we store the current value of x, compute the change from the equation, add the change to x, and loop.

```python
x = 100                      # initial condition: x at t = 0

for t in range(1, 9):        # time steps t = 1 through t = 8
    change = -(1/3) * x      # the equation: change in x is -1/3 of x
    x = x + change           # new x = old x + change
    print(t, round(x, 2))
```

- The line `change = -(1/3) * x` is the differential equation itself.
- The line `x = x + change` is the Euler step. It is the same rule as $x_t = x_{t-1} - \frac{1}{3}x_{t-1}$.
- The loop prints 66.67, 44.44, 29.63, 19.75, 13.17, 8.78, 5.85, and 3.9, the same values we computed by hand.
- Changing `1/3` to `3` in the change line would give the muscle equation instead of the fat equation. Nothing else in the program needs to change, since the method is identical and only the rule for change is different.

Section 6: Software 1.0

- **Software 1.0** is the traditional way of programming. A human programmer thinks through the problem and writes out every instruction by hand, line by line. The computer then follows those instructions exactly. Every decision the program makes was designed in advance by a person.
- A well-known black-and-white photograph from the early era of software shows a young programmer standing beside a stack of printed program listings that is roughly as tall as she is. Every page in that stack is code written by people.
    - The height of the stack shows how much hand-written instruction was needed for a single large system. Each line had to be planned, written, and checked by people.
    - A later color photograph shows the same programmer, decades older, receiving a medal at a White House ceremony, with the presidential seal on the podium. Hand-written software of this kind was important enough to earn national honors.
- Software 1.0 works well when people know exactly what steps to take. For tasks like recognizing a face among millions of pixels or recognizing a speech sound from a pressure wave, nobody can write out all the rules by hand. Neural networks handle this kind of task differently.

Section 7: What a Neural Network Does

- Two example problems show what a neural network is built to do.
    - In the first, we are given many pictures of faces, each made of **48 million pixels**, and asked to identify which of **1000 possible people** each face belongs to.
    - In the second, we are given **speech waves** and asked to identify which of **25 sounds** each speech wave corresponds to.
    - The system that can do both of these tasks is a **Neural Network (NN)**.

- We will not start with why it is called a neural network. Instead, we will define it by **what it does**, the same way we would recognize a duck by how it walks and how it swims. The behavior comes first, and the inner workings come later.
- **The job of a neural network is to take in a large set of numbers and return a probability distribution.** For an image, the numbers are the 48 million pixel values. For speech, the numbers are the values that make up the sound wave.
    - A **probability distribution** is a list of probabilities, one for each possible answer. Each probability says how likely it is that the input belongs to that answer.
    - For the speech task, the probability distribution covers the **25 sounds**, with one probability for each sound.
    - When this distribution is drawn as a graph, the probability p is on the vertical axis and the possible sounds are lined up along the horizontal axis, such as [a], [t], [p], [s], and so on, up to all the sounds. The height of the curve above each sound shows how likely the network thinks that sound is. The curve rises to a peak near one sound, then dips and rises a bit for others, then trails off toward the end.
- To make this concrete, suppose the network only had to decide among **5 vowels**. A probability distribution over them might be:

$$[.2,\ .2,\ .4,\ .2,\ 0]$$

- To read this list, we need to agree on the order of the vowels. The order is the vowel in **read**, the vowel in **rad**, the vowel in **rude**, the vowel in **Rud**, and the vowel in **rod**.
- The probabilities **must add up to 1**, which is 100%. We can check this:

$$.2 + .2 + .4 + .2 + 0 = 1$$

- This distribution says there is a **20% chance** the sound is the vowel in *read*, a **20% chance** it is the vowel in *rad*, a **40% chance** it is the vowel in *rude*, a **20% chance** it is the vowel in *Rud*, and a **0% chance** it is the vowel in *rod*.
- The largest probability, 40% for the vowel in *rude*, is the network's best guess. The distribution also shows how uncertain the network is. It spreads the rest of its belief over three other vowels and rules out one completely.

[CONTINUE]

Section 8: Inputs, Weights, and Learning by SGD

- The **input** of a neural network is a long list of numbers, also called a **signal** or a **vector**. An example is $[4,\ 1,\ -3,\ 2,\ 1,\ .4,\ 1,\ 2,\ -3,\ \ldots]$, and a real input usually has thousands of numbers. For speech, these numbers are the **pressure values of the speech wave**. Each number records the air pressure at one moment in time, so the list as a whole traces out the shape of the sound wave.
- The **output** of the network is a **decision probability distribution**. Given the signal, the network reports the probability of each possible object. For the five-vowel case, it gives one probability each for the vowels in *read*, *rad*, *rude*, *Rud*, and *rod*.
- A neural network turns a list of pressure numbers into a decision using its **knowledge**, which is stored in a large set of numbers called **weights**, written as w's.
    - The input signal vector is **combined with these weights**, and after many rounds of calculation, a decision about which object was detected comes out the other end.
    - The scale is enormous. Large networks have **several trillion w's** and carry out **billions of calculations** to reach a single decision.
    - The weights are what let the network make correct decisions. With good weights, the right answer gets a high probability. With bad weights, the output

is essentially a guess.

- The network is not given good weights. It has to find them, and it does so by **"faking it till it makes it."** The method is called **SGD**, which stands for **stochastic gradient descent**. "Stochastic" means the training examples are fed in one at a time in random order. "Gradient descent" means the weights are nudged, step by step, in whatever direction makes the network's mistakes smaller.
- The SGD learning procedure follows a fixed sequence of steps.
    1. **The network starts with random w's.** At this point it knows nothing, and its weights are just arbitrary numbers.
    2. **We give it a signal whose identity we know.** For example, we give it the pressure wave for the vowel in *Rud*.
    3. **Forward propagation.** The network passes the signal through its current weights, which at the start are useless, and produces a probability distribution. That distribution does not reflect reality. It might output:

$$[.4,\ .1,\ .2,\ .2,\ .1]$$

```
 - Reading this in the agreed order, the network says there is a 40% chance the
sound is the vowel in *read*, 10% for *rad*, 20% for *rude*, 20% for *Rud*, and 10%
for *rod*. Its top guess is *read*, which is wrong, and it gives the correct vowel
only 20%.
```

4. **We tell it the correct answer.** The network should have decided:

$$[0,\ 0,\ 0,\ 1,\ 0]$$

```
 - This is the vowel in *Rud* with 100% certainty and 0% for everything else. The
gap between what the network said and what it should have said is its **Error**,
also called its **Loss**. Here the Loss is large, since the network put most of its
probability on the wrong vowels.
```

5. **The network improves its w's to reduce the Loss.** Each weight is adjusted slightly in the direction that would have made the output closer to $[0,\ 0,\ 0,\ 1,\ 0]$.
6. **We give it another signal and repeat, billions of times.** Each repetition makes the weights a little better, and over billions of repetitions the random weights become weights that make correct decisions.

- This learning process is itself a **differential equation solved by stepping**, just like Euler's Method. The weights play the role of x. At each step, the Loss determines how the weights should change, the change is added to the current weights, and the process repeats. This is why **SGD** belongs on the list of real-world solutions to differential equations alongside flesh and room temperature. The Loss shrinks step by step toward a low, balanced value, much like x shrinking toward zero.
- None of this yet says **what calculations are actually carried out on the signal** when the input is combined with the weights. That calculation is the core of how a neural network works.

Section 9: Templatic Perception and Hierarchical Perception

- The goal is to understand **how a neural network uses its weights to process the input and make the right decision at the output.** For example, when the network is given a **[z] speech signal**, meaning a waveform or pressure waveform for the sound [z], the correct decision is **[z].** It should not decide [s], and it should not decide [a], or any other sound.
- One useful source of ideas is **how humans perceive speech, faces, and other objects.** Our brains contain **biological neural networks** that already solve these problems. If we

know how those biological networks perceive speech, we can use that knowledge to understand how artificial neural networks model the same task through artificial computation.

- Two basic theories of animal perception have shaped how neural networks are designed:
  - **Templatic Perception**, which is about comparing an input to stored patterns.
  - **Hierarchical Perception**, which is about processing an input in a series of stages.

Section 10: Templatic Perception

- **Templatic perception** is a theory that explains recognition as matching the input against stored patterns. Face perception is the clearest example, and the theory describes it as a sequence of steps.
  1. **The retina in the eye receives light information from the face.** This is the raw input, a pattern of light hitting the back of the eye.
  2. **The brain has a template stored in it for every face the person knows.** A template is a kind of stored picture, a remembered pattern for what each familiar face looks like.
  3. **The brain compares the input face with each of its stored face templates.** Every template is checked against what the eye is currently seeing.
  4. **The template with the highest similarity to the input face is what is perceived.** The best match wins.
  5. **If there is a name associated with the winning template, that name is the perceptual decision.** Recognizing someone means that the best-matching template carries that person's name.
- This theory has a direct implication for artificial intelligence: **the knowledge weights of a neural network are like perceptual templates.** The weights store the network's patterns for what each object should look or sound like.
- This connects to how learning begins. The network combines its current weights with the input, such as the pixels of a face, and produces a **Computed Output**. Before any learning has happened, the templates are random, so the Computed Output is wrong.
- The way a network's **weights, knowledge, and templates** are combined with the input is by **measuring the similarity between the weights and the input.** The network asks how closely the input matches each of its stored templates, just as the brain does in templatic face perception.

Section 11: Hierarchical Perception

- Psychologists and neuroscientists have known for a long time that **perception happens in stages, hierarchically.** The brain does not jump straight from raw input to a final decision.
  - **The input is turned into one new representation.**
  - **That representation is then turned into another representation.**
  - **That one is turned into another, and so on, until the output decision is made.**
  - A **representation** is the brain's way of encoding information at a given stage. Early representations are simple and close to the raw input, and later representations are more abstract and closer to a decision.
- The visual system of a primate brain, seen from the side with the front of the brain on the left and the back on the right, lays out this staged processing as a network of labeled areas connected by arrows.
  - Visual processing starts at the very back of the brain in **V1**, the **primary visual cortex**. This is the first cortical area to receive signals from the eyes. From V1, information passes to **V2** and **V3**, which sit just in front of it.

    - From these early areas, processing splits into two main routes: the **dorsal pathway**, running up and forward across the top of the brain (shaded blue), and the **ventral pathway**, running down and forward along the bottom of the brain (shaded red).
- The **ventral pathway** runs along the lower part of the brain, in the temporal lobe. It is often described as the pathway for identifying **what** an object is.
    - The chain of red arrows moves from **V1** to **V2**, then to **V4**, then to **TEO**, and finally to **IT**, the **inferior temporal cortex**.
    - Each area builds a more complex representation than the one before it. Early areas respond to simple features like edges, and IT, at the end of the chain, responds to whole objects such as faces. This makes the ventral pathway the most direct example of hierarchical perception leading to recognition.
    - A dashed arrow also runs directly between IT and V1, showing that some connections skip over intermediate stages.
- The **dorsal pathway** runs along the upper part of the brain, into the parietal and frontal lobes. It is often described as the pathway for **where** things are and **how** to act on them.
    - From V1, V2, and V3, blue arrows lead to **MT**/**MST**, the **middle temporal and medial superior temporal areas**, which process motion.
    - From there, arrows lead to a cluster of areas in the parietal lobe: **LIP** (lateral intraparietal area), **MIP** (medial intraparietal area), **VIP** (ventral intraparietal area), and **AIP** (anterior intraparietal area). These areas combine visual information with information about space and planned movements.
    - The parietal areas connect forward to the frontal lobe: **FEF** (frontal eye fields, which control eye movements), **PMd** (dorsal premotor cortex), and **PMv** (ventral premotor cortex), which help plan movements.
    - At the front of the brain is **PF**, the **prefrontal cortex**. PF receives input from the dorsal pathway, and a long purple arrow also connects PF with IT, linking the end of the ventral pathway to the front of the brain.
- Almost every arrow points in **both directions**. Information flows forward from early stages to later stages, and it also flows backward from later stages to earlier ones. Even so, the overall organization is a **hierarchy of stages**, with each area passing a transformed representation on to the next.

Section 12: Neural Networks as Templatic Hierarchical Perceivers

- **Neural networks are templatic hierarchical perceivers.** They combine both theories. At each stage, they compare the input against templates, and they do this over and over across many stages. The templatic part comes first.
- The key to templatic perception is that **an internal template of numbers is compared to an input set of numbers to find out how similar the template is to the input.**
    - Neural networks assume that the **input is a list, or vector, of numbers.**
    - They also assume that the **internal template weights are a list, or vector, of numbers.**
    - Recognition therefore comes down to comparing two lists of numbers to decide whether they are similar or not.
- Here is an example with three lists:

$$\text{Input} = [-1,\ 3,\ -2]$$

$$\text{Temp1} = [-2,\ 0,\ 3]$$

$$\text{Temp2} = [-2,\ 2,\ -1]$$

- The question is whether the Input is more similar to Template 1 or to Template 2.

- The tool for measuring similarity between two lists is the **inner product (IP)**. It is computed in two steps.
    1. **Multiply the two lists component-wise.** The first number of one list is multiplied by the first number of the other, the second by the second, and the third by the third.
    2. **Add all the products together.** The single number that results is the inner product.
- The inner product is read as a similarity score.
    - **If the IP is near 0, the lists are dissimilar.**
    - **If the IP is large and positive, the lists are similar.**
- For the Input and Template 1, the component-wise products are:

$$(-1)(-2) = 2$$

$$(3)(0) = 0$$

$$(-2)(3) = -6$$

- Adding the products:

$$2 + 0 + (-6) = -4$$

- The inner product is -4. It is small and negative, nowhere near a large positive value, so the Input is **not similar** to Template 1. The positive product from the first position is canceled by the negative product from the third position, since the two lists point in opposite directions there.
- For the Input and Template 2, the component-wise products are:

$$(-1)(-2) = 2$$

$$(3)(2) = 6$$

$$(-2)(-1) = 2$$

- Adding the products:

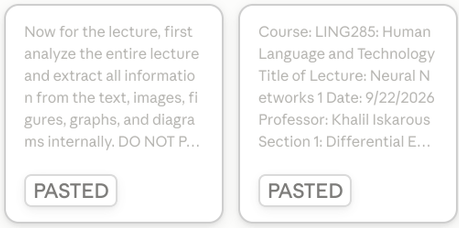
$$2 + 6 + 2 = 10$$

- The inner product is 10, which is large and positive. Every position contributes a positive product, since the Input and Template 2 have the same sign in all three positions. **The Input is much more similar to Template 2.**
- This is template matching done with arithmetic. A neural network holding Template 1 and Template 2 as weights would compute both inner products and find that Template 2 wins, with a score of 10 against -4. If Template 2 were the stored pattern for some sound or face, that would be the perceptual decision.

Section 13: Inference and Learning in Neural Networks

- A neural network works in two modes, **inference** and **learning**, and both rely on the inner product.
- **Inference** is how the network makes a decision. We give the system a signal vector, and its weights are inner-producted with it. This is the **template matching** step. The result becomes a new representation, which is then inner-producted with the next set of weights, and this repeats **over and over** across many stages. This is the **hierarchical perception** step.
    - Each stage takes the previous stage's output as its input, much as V1 feeds V2 and V4 feeds TEO in the brain.

    - The process runs step by step, **like Euler's algorithm.** Each step takes the current values, applies a fixed rule, and produces the next values, until the final probability distribution comes out at the end.
- **Learning** is how the network gets good weights. We give the system a signal vector **whose identity we already know** and ask it to infer that identity using its **initial random** w**'s.** With random weights, it cannot do this correctly, and its output does not match the correct answer.
    - The gap between its output and the correct answer is the Loss.
    - The w**'s are then improved through SGD**, nudged step by step in the direction that reduces the Loss.
    - Repeating this over many known examples gradually turns the random templates into templates that match real speech sounds and real faces, so that inference produces the right decision.



Course: LING285: Human Language and Technology
Title of Lecture: Neural Networks 1
Date: 9/22/2026
Professor: Khalil Iskarous

Section 1: Differential Equations and Dynamical Systems

- A **dynamical system** is something whose state changes over time according to a rule. A **differential equation** is the mathematical way of writing that rule. It does not state where the system is. It states how the system is changing, given where it is right now. For this reason, **differential equations are the mathematical expression of dynamical systems**.
- In **1676, Newton wrote a letter to Leibniz** and hid an idea inside an **anagram**, which is a scrambled set of letters that works as a coded message. The anagram would be revealed once Newton had fully proven the idea. This was how scientists of that time **established priority**, meaning proof of who had an idea first.
    - The secret was **6accdae13eff7i3l9n4o4qrr4s8t12ux**. Each number counts how many times the letter after it appears. For example, "6a" means six a's and "13e" means thirteen e's.
    - Unscrambled, the letters spell the Latin sentence **"Data aequatione quotcunque fluentes quantitates involvente fluxiones invenire, et vice versa."** It roughly means: given an equation involving any number of changing quantities, find their rates of change, and the reverse. Newton's word for a changing quantity was a **fluent**, and his word for its rate of change was a **fluxion**.
- **Vladimir Arnold** paraphrased Newton's secret completely: **the laws of nature are expressed by differential equations.** A law of nature is a rule for how things change, and a differential equation is how that rule is written.
- **Inference and learning in LLMs are differential equations.** When a large language model reaches a decision, and when it learns, it is following a rule for change applied step by step.

Section 2: The Viscous Spring and Negative Feedback

- A **viscous spring** returns to its resting position smoothly, slowing down as it gets closer, without bouncing. Its behavior is described by this equation:

$$\dot{x} = -\frac{1}{3}x$$

- The symbol x is the current value of the quantity. The symbol $\dot{x}$ is **just the change in x.**
- The equation says that **the change in x is $-\frac{1}{3}$ of its current value.**
- This is **negative feedback**, also called **balancing feedback**. The minus sign means the change always pushes x back toward zero. When x is positive, the change is negative, so x shrinks. The size of the push also depends on the size of x. A large x produces a large change, and a small x produces a small change, so the system moves fast at first and slows as it nears balance.
- The point of a differential equation is to take the equation together with an **initial condition**, the value of x at the start, and figure out x for all time. The complete description of x over time is called **the solution to the differential equation**.
    - The solution is a **phenomenon in the world**. Examples are **SGD**, **flesh**, and the **temperature in a room**. Each of these changes according to a rule, and the solution describes how it behaves over time.

Section 3: Euler's Method

- Let the initial condition be $x = 100$, so at time $t = 0$, x is 100. The equation says the change in x is $-\frac{1}{3}$ of its current value, and one third of 100 is about 33:

$$\dot{x} = -\frac{1}{3}(100) \approx -33$$

- Starting at 100 and losing about 33 brings x to about 66:

$$x_1 = 100 - 33.33 = 66.67$$

- This calculation can be written as a general rule. The new value of x equals the previous value minus one third of the previous value:

$$x_t = x_{t-1} - \frac{1}{3}x_{t-1}$$

- Here x_t is the value at the current time step and x_{t-1} is the value at the step before.
- Now x is about 66, so we plug it into the equation again. One third of 66.67 is about 22.22, and we take that away from 66.67:

$$x_2 = 66.67 - 22.22 = 44.44$$

- One third of 44.44 is about 14.81:

$$x_3 = 44.44 - 14.81 = 29.63$$

- One third of 29.63 is about 9.88:

$$x_4 = 29.63 - 9.88 = 19.75$$

- When x is plotted against t from $t = 0$ to $t = 8$, the points start at 100 and drop steeply over the first few steps. Each drop is smaller than the one before, so the points level off and move closer and closer to the t axis. The value falls quickly while x is large and slowly once x is small.
- This procedure is **Euler's Method**. At each time step, the equation gives the change based on the current value of x. Adding that change to the current value gives the next value. Repeating this builds the whole solution one step at a time. **Differential equations are solved this way to predict what the world is like using arithmetic.**

Section 4: Fat and Muscle

- The number multiplying x sets **how fast** x returns to zero. Two balancing-feedback equations with different numbers produce curves with the same shape but different speeds.
- The first equation produces a slow return:

$$\dot{x} = -\frac{1}{3}x$$

- Its curve starts high and declines gradually, staying above zero for a long time before flattening near the t axis. This curve is labeled **fat**.
- The second equation produces a fast return:

$$\dot{x} = -3x$$

- The change is three times the current value instead of one third of it, which makes the push toward zero nine times stronger at any given x. Its curve drops almost straight down right after the start and then runs flat along the t axis. This curve is labeled **muscle**.
- Both fat and muscle behave as **viscous springs with negative feedback**. The only difference is the size of the number in the equation. A small number gives a slow return, and a large number gives a fast return.

Section 5: Euler's Method in Python

- Euler's Method is repeated arithmetic, so it can be written as a short Python loop. The program stores x, computes the change from the equation, adds the change to x, and repeats.

```python
x = 100                         # initial condition at t = 0
for t in range(1, 9):           # time steps t = 1 through 8
    change = -(1/3) * x         # the change in x is -1/3 of x
    x = x + change              # x_t = x_{t-1} - (1/3) x_{t-1}
    print(t, round(x, 2))
```

- The line `change = -(1/3) * x` is the differential equation, and the line `x = x + change` is the Euler step.
- The loop prints 66.67, 44.44, 29.63, 19.75, 13.17, 8.78, 5.85, and 3.9.
- Replacing `1/3` with `3` turns the fat equation into the muscle equation.

Section 6: Software 1.0

- **Software 1.0** is software written by people, instruction by instruction. Every step the computer follows was planned and written out by a programmer.
  - A black-and-white photograph shows a young programmer standing beside a stack of printed code listings about as tall as she is. The whole stack is hand-written instructions for a single software system.
  - A later photograph shows the same programmer, decades older, receiving a medal at a ceremony with the presidential seal on the podium.

Section 7: What a Neural Network Does

- Two tasks define the problem a neural network solves.
  - Given many pictures of faces, each made of **48 million pixels**, identify which of **1000 possible people** each face belongs to.

    - Given speech waves, identify which of **25 sounds** each speech wave corresponds to.
    - A **Neural Network (NN)** can do both. Here it is defined by what it does rather than by why it is called a neural network.
- **The job of an NN is to take a large set of numbers, such as the 48 million pixels of one image or the numbers of a speech wave, and return a probability distribution.** A **probability distribution** is a list with one probability for each possible answer.
    - For speech, the probability distribution covers all **25 sounds**. Graphed with probability p on the vertical axis and the sounds ([a], [t], [p], [s], and so on) along the horizontal axis, the curve rises to a peak over the most likely sound and stays lower over the others.
- Suppose there were only **5 vowels**, ordered as the vowels in **read, rad, rude, Rud, rod**. One possible probability distribution is:

$$[.2,\ .2,\ .4,\ .2,\ 0]$$

- **The probabilities must sum to 1, which is 100%:**

$$.2 + .2 + .4 + .2 + 0 = 1$$

- This distribution says there is a **20%** chance the sound is the vowel in *read*, **20%** for the vowel in *rad*, **40%** for the vowel in *rude*, **20%** for the vowel in *Rud*, and **0%** for the vowel in *rod*.

Section 8: Inputs, Weights, and Learning by SGD

- The **input** of an NN is a list of numbers called a **signal** or **vector**, such as $[4,\ 1,\ -3,\ 2,\ 1,\ .4,\ 1,\ 2,\ -3,\ \ldots]$. Real inputs usually contain thousands of numbers. For speech, these are the **pressure numbers of the speech wave**.
- The **output** is a **decision probability distribution**. Given the signal, the network reports the probability of each object, such as each of the vowels in *read, rad, rude, Rud,* and *rod*.
- The network makes its decision using **knowledge stored in a large set of weights**, written as w's. The input signal vector is combined with these weights, and a decision about the detected object is eventually produced. Large networks have **several trillion w's** and perform **billions of calculations**.
- The network finds good weights by **"faking it till it makes it"** through **SGD** (stochastic gradient descent), which gradually adjusts the weights to reduce the network's errors. The procedure has a fixed sequence.
    1. **Start with random w's.**
    2. **Give the network a signal whose identity is known**, for example the vowel in *Rud*.
    3. **Forward propagation.** The network uses its untrained weights and produces a probability distribution that does not reflect reality:

$$[.4,\ .1,\ .2,\ .2,\ .1]$$

```
 - This gives the vowel in *read* the highest probability at 40% and the correct vowel, *Rud*, only 20%.
```

4. **Tell the network the correct answer.** It should have decided:

$$[0,\ 0,\ 0,\ 1,\ 0]$$

```
 - The difference between its output and the correct answer is a large **Error**,
```

```
also called **Loss**.
```

5. **The network improves its w's to reduce the Loss.**
6. **Give it another signal and repeat billions of times.**

- SGD is one of the real-world solutions to differential equations, alongside flesh and room temperature. Learning follows a rule for change applied step by step, and the Loss is reduced at each step, the same stepping process as Euler's Method.

Section 9: Templatic Perception and Hierarchical Perception

- The goal is to know how an NN uses its weights to turn an input into the right decision. Given a **[z] speech signal**, meaning a pressure waveform of [z], the correct decision is **[z]**, not [s] and not [a].
- Human perception of speech and faces is carried out by **biological neural networks**. Knowing how these perceive speech helps explain how NNs model the same task through artificial computation. Two basic theories of animal perception inform NN design: **Templatic Perception** and **Hierarchical Perception**.

Section 10: Templatic Perception

- **Templatic perception** is a theory of how face perception works. It proceeds in steps.
    1. **The retina in the eye receives light information from the face.**
    2. **The brain has a template, a stored picture of sorts, for every face the person knows.**
    3. **The brain compares the input face with each of its stored face templates.**
    4. **The template with the highest similarity to the input face is what is perceived.**
    5. **If a name is associated with the winning template, that name is the perceptual decision.**
- For AI, this theory implies that **the knowledge weights of an NN are like perceptual templates**.
    - An NN combines its current weights with the input face pixels to produce a **Computed Output**. Before any learning, that output is wrong.
    - **An NN's weights, which are its knowledge and templates, are combined with the input by determining the similarity between the weights and the input.**

Section 11: Hierarchical Perception

- Psychologists and neuroscientists have long known that **perception is done in stages, hierarchically**. The input becomes one new representation. That representation becomes another, and that one becomes another, until the output decision is made. A **representation** is the form the information takes at a given stage.
- The visual areas of the primate brain are organized as this kind of staged hierarchy, with the front of the brain at the left and the back at the right.
    - Processing begins in **V1**, the primary visual cortex, at the back of the brain. It then connects to **V2** and **V3**.
    - The **ventral pathway** runs along the lower part of the brain. Its connections run from **V1** to **V2**, then to **V4**, then to **TEO**, and then to **IT**, the inferior temporal cortex. A dashed connection also links **IT** directly with **V1**.
    - The **dorsal pathway** runs along the upper part of the brain. Its connections run from **V1**, **V2**, and **V3** to **MT/MST**. From there they reach the parietal areas **LIP, MIP, VIP,** and **AIP**, and then the frontal areas **FEF**, **PMd**, **PMv**, and **PF** (prefrontal cortex).

- **PF** is also connected directly with **IT**, which links the end of the ventral pathway to the front of the brain.
- Most connections run in both directions between areas. Even so, each area receives a representation from one stage and passes a new representation to the next.

Section 12: NNs as Templatic Hierarchical Perceivers

- **NNs are templatic hierarchical perceivers.** In templatic perception, **an internal template of numbers is compared to an input set of numbers to find out how similar the template is to the input.** NNs assume that the **input is a list, or vector, of numbers** and that the **internal template weights are also a vector of numbers**.
- Take one input and two templates:

$$\text{Input} = [-1,\ 3,\ -2]$$

$$\text{Temp1} = [-2,\ 0,\ 3]$$

$$\text{Temp2} = [-2,\ 2,\ -1]$$

- The question is whether the Input is more similar to Template 1 or Template 2.
- Similarity between two lists is measured with the **inner product (IP)**, which is computed in two steps.
  1. **Multiply the two lists component-wise**, so the first number is multiplied by the first number, the second by the second, and the third by the third.
  2. **Add all the products.**
  - **If the IP is near 0, the lists are dissimilar. If the IP is large and positive, the lists are similar.**
- For Input and Template 1, the component-wise products are:

$$(-1)(-2) = 2$$

$$(3)(0) = 0$$

$$(-2)(3) = -6$$

- Adding the products gives the inner product:

$$2 + 0 + (-6) = -4$$

- The IP is -4, which is not large and positive, so the Input is **not similar** to Template 1.
- For Input and Template 2, the component-wise products are:

$$(-1)(-2) = 2$$

$$(3)(2) = 6$$

$$(-2)(-1) = 2$$

- Adding the products gives the inner product:

$$2 + 6 + 2 = 10$$

- The IP is 10, which is large and positive, so **the Input is more similar to Template 2**. In template matching, Template 2 wins.

Section 13: Neural Networks: Inference and Learning

- **Inference** is how an NN makes a decision. The system is given a signal vector, and the w's are inner-producted with it, which is **template matching**. The result is inner-producted with further weights over and over, which is **hierarchical perception**. The computation proceeds step by step, **like Euler's algorithm**, until the output probability distribution is produced.
- **Learning** is how an NN gets its weights. The system is given a signal vector whose identity is known and is asked to infer that identity using its initial random w's. It cannot do this correctly, so **the w's are improved through SGD**.